

COLEÇÃO SE EU IA FOSSE HUMANO

Se Eu IA Fosse Humano - Apresentação

Coleção Se Eu IA Fosse Humano

Capa

Nexus Affil'IA'te — IOAID - SaaS

INFRAESTRUTURA OPERACIONAL DE INTELIGÊNCIA DISTRIBUÍDA (SaaS
Early-Stage)

Organismo SaaS /AI Native - Ecossistema de Marketing
Afiliaados adotando o "SHO" - Sistema Híbrido de
Orquestração e buscando alcançar o nível de "AOI"
Autonomous Operacional Intelligence.

Gestão x Operacional O usuário/peer se cadastra na
plataforma e ajusta as funcionalidades operacionais e
skills dos Agentes IA autônomos, promovendo ações em
estrutura AI Operational Network.

O processo segue um modelo de operação singular, operando através de
uma arquitetura de alta integridade no modelo "Full Autonomous
Runtime".

O Sistema Legacy aplicado no desenvolvimento, implementou uma arquitetura do sistema original fusionado do legado PHP com stack moderna React/TypeScript.

Documentação Canônica

⚠ **IMPORTANTE:** Para uma visão completa e atualizada do sistema, consulte a **Documentação Canônica**:

[DOCUMENTAÇÃO CANÔNICA](#)

[ÍNDICE DE DOCUMENTAÇÃO](#) - Navegação centralizada por todos os documentos do projeto

Esta documentação centraliza todas as informações do sistema em um único documento de referência, incluindo:

- Visão Geral do Sistema
- Sistema MMN (Comissões e Carreiras)
- Painel Administrativo e RBAC
- Painel do Afiliado
- Agentes e Skills
- Marketplace Nexus
- Nível de Autonomia
- Potencial de Mercado
- Stack Tecnológica Completa
- Guia de Início Rápido
- Roadmap e Conformidade

Análise Técnica Detalhada: [Análise Técnica Fundamentalista v2.0](#)

Para navegar toda a documentação de forma organizada, consulte o [ÍNDICE DE DOCUMENTAÇÃO](#).

Status do Projeto

Stage

MVP+

Conformidade

92-95%

License

MIT

Agentic

Layer

XP/Carreiras

Implemented

v1.2.0

2026-05-25

Progresso das Fases

FASE 1-4		FINALIZADA
FASE 5		FINALIZADA
FASE 6		FINALIZADA
FASE 7		FINALIZADA (White-Label)
FASE 8		FINALIZADA (Beta Launch)
FASE 9		FINALIZADA (GA Launch)
FASE 10		PLANEJADO
FASE 10		PLANEJADO (v1.3.0)

Roadmap Detalhado

Consulte [fases/ROADMAP_FASES.md](#) para informações completas sobre cada fase.

Fase	Descrição	Status	Referência
Fases 1-4	Core Backend e Frontend	Finalizado -	
Fase 5	Sistema MMN	Finalizado -	
Fase 6	Agentes IA + Runtime	Finalizado -	
Fase 7	White-Label Module	Finalizado	fase7/
Fase 8	Beta Launch Program	Finalizado	fase8/
Fase 9	GA Launch	Finalizado	fase9/
Fase 10	Estabilização + Integrações	Planejado	fases/FASE10_ROADMAP.md

Fase 10 - Próximas Funcionalidades

Consulte [fases/FASE10_ROADMAP.md](#) para o roadmap completo da Fase 10, incluindo:

- **Epic 10.1:** Mobile Expo - Estabilização Completa
- **Epic 10.2:** Integração PIX - Pagamentos Instantâneos
- **Epic 10.3:** Firebase Auth - Autenticação Completa
- **Epic 10.4:** WhatsApp API - Automação de Mensagens
- **Epic 10.5:** Performance e Cache
- **Epic 10.6:** Observabilidade e Monitoring
- **Epic 10.7:** Multi-tenancy Foundation
- **Epic 10.8:** Segurança e Compliance

Aviso: Este projeto está em desenvolvimento ativo. Algumas funcionalidades descritas neste documento estão em implementação ou planejadas para fases futuras.

Revisão Atual do Sistema

Revisão Técnica Consolidada

[Revisão Técnica Consolidada v1.0](#) - Visão geral completa do estado do sistema

A revisão técnica consolidada do estado atual do repositório está em: A
revisão técnica detalhada do estado atual do repositório está em:

- [docs/repository-review/ANALISE_TECNICA_SISTEMA_ATUAL.md](#)
- [docs/repository-review/RESUMO_EXECUTIVO_SISTEMA_ATUAL.md](#)
- [docs/repository-review/CONSOLIDACAO_DOCUMENTAL_FASE2.md](#)
- [docs/repository-review/ORQUESTRADOR_DASHBOARD_AVALIACAO.md](#)
- [docs/repository-review/MAPA_ROTAS_E_UNIFICACAO_FRONTEND.md](#)
- [docs/repository-review/README.md](#)
- [docs/README.md](#)

Backoffice Admin MMN AI-to-AI

Para iniciar o desenvolvimento do Backoffice Admin, a trilha oficial desta etapa está em:

- [docs/admin-backoffice/README.md](#)
- [docs/admin-backoffice/PLANO_EXECUCAO_EM_FASES.md](#)
- [docs/admin-backoffice/BACKLOG_INICIAL.md](#)
- [docs/admin-backoffice/INVENTARIO_ATUAL.md](#)
- [docs/admin-backoffice/FASE_1_ENTREGA_INICIAL.md](#)
- [docs/admin-backoffice/FASE_2_NUCLEO_OPERACIONAL.md](#)
- [docs/admin-backoffice/FASE_2_COMPLEMENTO_OPERACIONAL.md](#)
- [docs/admin-backoffice/ENTREGA_APROVACOES_ADMINISTRATIVAS.md](#)
- [docs/admin-backoffice/ENTREGA_COMISSOES_NAMESPACE_DEDICADO.md](#)
- [docs/admin-backoffice/ENTREGA_AUDITORIA_E_CONSOLIDACAO_FINANCEIRA.md](#)
- [docs/admin-backoffice/ENTREGA_AGENDAMENTOS_CRON_ADMIN.md](#)
- [docs/admin-backoffice/ENTREGA_CRON_DISPATCHER_BULLMQ.md](#)
- [docs/admin-backoffice/ENTREGA_CRON_HISTORY_SYNC.md](#)
- [docs/admin-backoffice/ENTREGA_SLA_CRON_BACKOFFICE.md](#)
- [docs/admin-backoffice/ENTREGA_ALERTAS_CRON_BACKOFFICE.md](#)
- [docs/admin-backoffice/ENTREGA_ALERTAS_CRON_PERSISTENCIA.md](#)
- [docs/admin-backoffice/ENTREGA_HISTORICO_ALERTAS_CRON_BACKOFFICE.md](#)

Atualizações Recentes do Repositório

(2026-05-25)

Fase 9 — GA Launch Program (Lançamento Geral)

A fase de **Lançamento GA** foi concluída com sucesso, disponibilizando a plataforma para o público geral:

- **Landing Page API:** Hero, features, pricing, testimonials, FAQ, changelog
- **Documentation System:** Guides, API reference, search functionality
- **Support Ticket System:** Tickets, SLA monitoring, knowledge base
- **Community Hub:** Forums, events, showcase, badges, leaderboards

Referências: - [fase9/README.md](#) - [fase9/SPEC.md](#)

Fase 8 — Beta Launch Program

Programa completo de lançamento beta com todas as funcionalidades:

- **Beta Tester Management:** CRUD, status tracking, invitations
- **Feedback System:** Submission, voting, categories, sentiment analysis
- **Bug Tracking:** Severity levels, status workflow, assignee tracking
- **Analytics Dashboard:** KPIs, engagement metrics, retention tracking

Referências: - [fase8/README.md](#) - [fase8/SPEC.md](#)

Fase 7 — White-Label Module

Módulo de white-label completo para multi-inquilino:

- **Instance Management:** CRUD, API key auth, rate limiting
- **Branding Engine:** Logos, colors, fonts, custom domains
- **Domain Management:** Subdomains, SSL certificates
- **Tenant Billing:** Plans, usage tracking, invoices

Referências: - [fase7/README.md](#) - [fase7/SPEC.md](#) - [fase7/FASE7_WHITE LABEL_MODULE.md](#)

Fase 6 — Revisão e Otimização

Consulte [fases/FASE6_REVISAO.md](#) para detalhes.

Atualizações Anteriores (2026-05-24)

Continuação da Fase Beta — Domains, Event Bus e trilha de auditoria

A continuidade da **Fase Beta — Transição MMN** avançou do endurecimento de infraestrutura para uma base mais modular e orientada a domínio:

- criada a camada `backend/src/domains/` como **anti-corruption layer** para os domínios `affiliate`, `commissions`, `marketplace`, `agent-runtime`, `billing`, `cron`, `xp` e `auth`
- o `backend/src/appRouter.ts` passou a consumir a nova camada em domínios priorizados da transição, reduzindo o acoplamento direto aos routers legados
- o Event Bus foi ligado a fluxos reais do runtime (`mmn.registerAffiliate`, `commissions.updateStatus`, `commissions.approveBatch`, `marketplaceSyncWorker`, `agentRuntime.generate` e `agentRuntime.generateBatch`)
- adicionado `backend/src/_core/events/auditSubscribers.ts` e registro automático no bootstrap do backend para gerar trilha estruturada mínima de auditoria de eventos
- adicionados testes unitários para `EventBus` e `healthRouter`
- o domínio `commissions` passou a extrair `types.ts`, `repository.ts` e `service.ts`, iniciando a migração real da lógica de negócio para dentro de `backend/src/domains/`
- o domínio `affiliate` recebeu `types.ts` e `service.ts` com `registerAffiliate` e erros tipados; o `mmnRouter` foi reduzido a uma camada de transporte que injeta dependências do banco no `service`

- o domínio marketplace recebeu `types.ts` , `repository.ts` e `service.ts` ; o `marketplacesRouter` agora delega conexão, desconexão, listagem, enfileiramento de sync e normalização das respostas de catálogo ao domínio
- o domínio agent-runtime recebeu `types.ts` , `repository.ts` e `service.ts` ; o `agentRuntimeRouter` agora delega perfil, geração, batch, bump de performance e auditoria ao domínio, incluindo publicação explícita de `AgentSessionFailed` em falhas de geração
- o domínio billing recebeu `types.ts` , `repository.ts` e `service.ts` ; o `billingRouter` agora delega leitura, listagem, criação, atualização de status, histórico, estatísticas e confirmação de pagamento ao domínio, com publicação de `InvoicePaid` , `InvoiceOverdue` e `PaymentProcessed`
- o domínio cron recebeu `types.ts` , `repository.ts` e `service.ts` ; o `cronRouter` agora delega listagem, CRUD, histórico, estatísticas, configurações, próximas execuções e validação de expressão cron ao domínio, com erros tipados `CronJobNotFoundError`
- adicionado `scripts/validate-beta-structure.mjs` com atalho `npm run verify:beta-structure` para checagem estrutural rápida da Fase Beta
- documentação consolidada em `docs/validation-reports/FASE_BETA_CONTINUATION.md`

Referências:

- [backend/src/domains/README.md](#)
- [backend/src/_core/events/auditSubscribers.ts](#)
- [docs/validation-reports/FASE_BETA_CONTINUATION.md](#)

Atualizações Recentes do Repositório

(2026-05-23)

Fase 6 — Agentes IA conectados ao runtime de LLM

A camada de **Agentes IA** ganhou um router unificado (`trpc.agentRuntime.*`) que integra agente + upgrades/skills + LLM em um único pipeline com auditoria:

- novo `backend/src/routers/agentRuntimeRouter.ts` expondo `getProfile`, `generate`, `generateBatch`, `bumpPerformance` e `registerAction`
- `generate` respeita a `contentStrategy` do usuário (plataforma, tom e público) e persiste auditoria em `session_audit` para a aba "Últimas ações"
- exporta `InsertUpgrade` e `InsertAgentUpgrade` em `database/schemas/schema-final.ts` para alinhar tipagem dos routers de upgrades
- registrado em `appRouter.bootstrap.status.routers.agentRuntime` para verificação de presença

Mobile conectado ao backend: `mobile/app/(tabs)/agent.tsx` foi reescrito para consumir `trpc.agentRuntime.getProfile`, alternar estratégia via `trpc.agents.configure`, ativar/desativar status do agente e gerar conteúdo em tempo real via `trpc.agentRuntime.generate` (com seleção de plataforma, tópico e exibição do resultado da IA).

Referências:

- [backend/src/routers/agentRuntimeRouter.ts](#)
- [backend/src/appRouter.ts](#)
- [mobile/app/\(tabs\)/agent.tsx](#)

Fase 7 — Marketplace com fila de sync, tendências e testes

A trilha de **Marketplaces** foi consolidada com execução real de sincronização via fila e cobertura de testes isolados:

- `backend/src/services/syncMarketplaceProducts.ts` agora suporta sync global, por marketplace ou por conta, persiste histórico, atualiza tendências (`product_trends`) e margens (`affiliate_margins`)
- `backend/src/workers/marketplaceSyncWorker.ts` deixou de retornar mocks fixos e passou a executar o serviço real de sincronização
- `backend/src/routers/marketplacesRouter.ts` agora enfileira sync manual com `jobId` , `marketplace` e `accountId` , em vez de apenas marcar status
- padronização do identificador `mercado_libre` entre router, fila, scheduler e cron dispatcher para evitar divergências entre execução manual e automática
- `tests/unit/marketplaces.test.ts` foi reescrito com mocks estáveis de DB/helpers/queue e agora valida conexão de contas, sync manual, produtos recomendados, analytics e margens agregadas

Validação local: `npm --workspace backend run build` `npx vitest run tests/unit/marketplaces.test.ts` .

Mobile Expo — trilha de estabilização em andamento

O workspace `mobile/` avançou na estabilização do fluxo de autenticação e tema para suportar a entrega MVP+:

- carregamento de variáveis de ambiente no Expo via `mobile/scripts/load-env.js`
- adição das constantes compartilhadas de OAuth e tema (`mobile/constants/oauth.ts` , `mobile/constants/theme.ts` , `mobile/lib/_core/theme.ts`)
- reescrita do callback OAuth em `mobile/app/oauth/callback.tsx`

- simplificação do layout raiz do Expo Router em `mobile/app/_layout.tsx` para reduzir a superfície do erro de export estático
- consolidação do provider global de tema em `mobile/lib/theme-provider.tsx`
- integração do toggle real de tema e redirecionamento de logout em `mobile/app/(tabs)/profile.tsx`

Bloqueio atual: ainda falta validar o build web do Expo após as correções, porque a trilha anterior falhava com o erro `Objects are not valid as a React child` durante a exportação estática da rota de login.

Atualizações Recentes do Repositório (2026-05-22)

Packs / Marketplace de Skills do Agente IA

Nova seção completa para aquisição de pacotes de skills para agentes IA autônomos:

- `schema packs` e `agent_packs` adicionados ao banco de dados principal
- router `trRPC packs.*` com 5 endpoints (`listAvailable`, `listMine`, `purchasePack`, `cancelPack`, `getPackDetails`)
- página `/packs` com design dark premium, 8 packs pré-configurados, filtros por categoria e badges visuais
- link "Pacotes / Skills" integrado à sidebar do `DashboardLayout`

Referências:

- [backend/src/routers/packsRouter.ts](#)
- [frontend/src/pages/PacksMarketplace.tsx](#)

DashboardLayout — Navegação Completa e Corrigida

Refatoração completa da sidebar do painel do afiliado:

- links corrigidos: `/dashboard` (era `/`) e `/agents` (era `/agent`)

- 20+ links organizados em 4 grupos: Geral, Agente IA, Marketing, Loja & Operações
- navegação com destaque do item ativo (estado real baseado em `useLocation`)
- uso de `setLocation` (wouter) no lugar de `<a href>` para evitar reloads
- dropdown do usuário com links funcionais para Perfil, Agente IA e Painel Admin (condicional)

Comissões — Página com Dados Reais

Substituição do placeholder "Seção em Desenvolvimento" por UI funcional:

- cards de KPI: Total Gerado, Pendentes, Confirmadas, Pagas via `trpc.commissions.getStats`
- tabela paginada via `trpc.commissions.list` com filtro por status
- distribuição por nível MMN (`byLevel`)
- estados de loading, erro e vazio tratados corretamente

Referências:

- [frontend/src/pages/DashboardLayout.tsx](#)
- [frontend/src/pages/Commissions.tsx](#)

Backoffice Admin consolidado por domínio

O Backoffice Admin avançou com uma trilha incremental já refletida no frontend, backend e documentação:

- aprovações administrativas integradas ao painel com rota dedicada
- aprovação em lote e indicadores de SLA na fila de aprovações
- migração de comissões para o namespace dedicado `trpc.commissions.*`
- reforço de auditoria e consolidação visual da fila financeira entre aprovações, comissões e pagamentos

Documentos de referência:

- [docs/admin-backoffice/ENTREGA_APROVACOES_ADMINISTRATIVAS.md](#)
- [docs/admin-backoffice/ENTREGA_COMISSOES_NAMESPACE_DEDICADO.md](#)
- [docs/admin-backoffice/ENTREGA_AUDITORIA_E_CONSOLIDACAO_FINANCEIRA.md](#)

Automação Cron, central administrativa e saneamento do backend

O backend e o Backoffice Admin avançaram juntos sobre o domínio Cron:

- implantação do módulo de automação Cron com router, scheduler e estrutura de histórico/configuração
- central administrativa completa em `/admin/schedules` ligada ao `trpc.cron.*` com CRUD de jobs (criar, editar, remover, executar agora, pausar/ativar)
- aplicação de templates pré-definidos via `cron.getTemplates` baseados em `CRON_JOB_CONFIGS`
- painel lateral de configurações globais do domínio Cron (timezone, canal de alertas, janela de manutenção)
- novo dispatcher Cron ↔ BullMQ (`backend/src/services/cronDispatcher.ts`) que conecta cada `jobType` à fila correta, com handlers inline para jobs curtos e fallback genérico
- `cron.runNow` agora executa o job de verdade (não apenas registra) com status final, duração real e metadados de despacho persistidos em `cron_job_history`
- sincronização automática do despecho BullMQ de volta para `cron_job_history` via helper `cronHistorySync` integrado aos 5 workers (`commission` , `content` , `marketplace` , `order` , `withdrawal`)
- camada de SLA do domínio Cron com snapshot backend (`cronSlaIndicators.ts`) e visualização no `AdminSchedules.tsx` , cobrindo sucesso 7d/30d, p95, falhas consecutivas e jobs travados

- sistema de alertas operacionais (`cronAlerts.ts`) integrado ao scheduler, com reavaliação automática a cada 5 min, persistência dedicada em `cron_alerts` , dedup multi-instância por chave/cooldown, notificações para admins, histórico de incidentes e MTTA/MTTR no `AdminSchedules.tsx`
- nova camada `cronAlertHistory.ts` com backlog paginado, filtros por estado/severidade/reconhecimento/tipo/jobType e snapshot executivo de incidentes resolvidos
- novo drilldown `cronAlertContext.ts` cruzando cada incidente com `cron_job_history` , jobs impactados e a central administrativa de logs, com navegação contextual entre `/admin/schedules` e `/admin/logs`
- saneamento da camada compartilhada de tRPC no frontend (`useTRPC`) e evolução do `ExecutionLogs.tsx` para aceitar filtros contextuais por query string

Referências úteis:

- [backend/src/routers/cronRouter.ts](#)
- [backend/src/services/cronScheduler.ts](#)
- [frontend/src/pages/AdminSchedules.tsx](#)
- [database/schemas/schema-cron.ts](#)
- [CHANGELOG.md](#)

Stack Tecnológica

Categoria	Tecnologia	Versão
Frontend Web	React 18 + Vite + wouter (router) + TailwindCSS + TanStack Query	^18.3.1 / ^6.0.7
Backend	Node.js + TypeScript + tRPC v11	^22.10.0
Banco de Dados	MySQL (Drizzle ORM) + Redis + BullMQ	^0.38.4 / ^5.28.2
Mobile		0.81.5 / ~54

Categoria	Tecnologia	Versão
	React Native + Expo Router (diretório mobile/)	
IA	Google Genkit (Gemini) + OpenAI	^1.0.0 / ^4.77.0
Auth	JWT (Firebase/NextAuth no roadmap)	-

Avanços Estruturais Consolidados

Migração Legacy → Sistema Oficial

Funcionalidade	Status	Descrição
Newsletter System	Migrado	Subscribe/Unsubscribe/List com endpoints tRPC
CMS Pages	Migrado	CRUD de páginas dinâmicas com meta tags
Billing System	Migrado	Faturas, itens e histórico de cobrança
Database Schemas	Criados	Tabelas para newsletters, cms_pages, invoices

Sistema de XP/Carreiras Implementado

Componente	Status	Descrição
Schema de Carreiras	Implementado	27 níveis organizados em 5 categorias
Cálculo de XP	Implementado	XP por vendas, comissões e bônus

Componente	Status	Descrição
Progressão Automática	Implementado	Cálculo de nível baseado em XP total
Leaderboard	Implementado	Top 10 afiliados por XP
Histórico de XP	Implementado	Transações detalhadas
Dashboard com Métricas Reais	Implementado	Dados reais do banco de dados

Camada Agentic Implementada

Componente	Status	Descrição
Persistência de Sessões	Implementado	Gradual para sessões e memória agentic
Monitoramento	Implementado	Camada de monitoramento e orquestração
Orquestração Multi-Agente	Implementado	Infraestrutura de coordenação
Logs de Auditoria	Implementado	Rastreamento completo de operações

Como Iniciar

1. Preparação

Pré-requisitos validados:

- Node.js 20+
- npm 10+

- Docker Desktop ou Docker Engine (opcional, para MySQL/Redis locais)

```
git clone https://github.com/Nexus-HUB57/MMN_AI-to-AI.git
cd MMN_AI-to-AI

# Instalação do monorepo (raiz + workspaces)
npm install

# Se houver problemas com workspaces npm, instale manualmente
cd backend && npm install && cd ..
cd frontend && npm install && cd ..
cd mobile && npm install && cd ..
```

Nota: Em alguns ambientes, o npm workspaces pode apresentar limitações. Se `npm install` não instalar as dependências dos workspaces, execute `npm install` diretamente em cada diretório (`backend/` , `frontend/` , `mobile/`).

2. Infraestrutura (Docker)

```
npm run infrastructure:up      # docker compose up -d
npm run infrastructure:logs    # acompanhar logs
npm run infrastructure:down    # derrubar containers
```

3. Banco de Dados

```
npm run db:generate           # drizzle-kit generate
npm run db:migrate             # drizzle-kit migrate
npm run db:push                 # drizzle-kit push (para desenvolvimento)
```

4. Variáveis de Ambiente

Copie `.env.example` para `.env` e preencha:

- `DATABASE_URL` → string MySQL
- `REDIS_URL` → `redis://localhost:6379`
- `OPENAI_API_KEY` , `JWT_SECRET` , `MYSQL_ROOT_PASSWORD` , `PORT`

5. Execução em Desenvolvimento

```
# Frontend + Backend juntos
npm run dev

# Separadamente:
npm run dev:frontend    # Vite dev server (porta 5173)
npm run dev:backend     # tsx watch do backend/src/index.ts
npm run dev:mobile      # Expo dev server

# Workers BullMQ
npm --workspace backend run worker:content
npm --workspace backend run worker:commissions
npm --workspace backend run worker:marketplace
npm --workspace backend run worker:orders

# Genkit dev (Gemini)
npm run genkit:dev
```

6. Build de Produção

```
npm run build
npm run start
```

Funcionalidades Implementadas

Funcionalidades Core

Funcionalidade	Status	Descrição
Stack Tecnológica	Completo	React + Vite + tRPC + TailwindCSS + Drizzle + MySQL + Redis + BullMQ
Autenticação JWT	Funcional	Contexto tRPC com JWT implementado
Sistema MMN Básico	Funcional	Comissões em cascata até 15 níveis, compressão dinâmica
Marketplaces	Parcial	Mercado Livre, Shopee, Hotmart integrados
Roteador LLM	Funcional	Google Genkit (Gemini) + OpenAI
Content Generation	Parcial	Textos, variações, hashtags, sentimento
Dropshipping	Funcional	Pedidos, tracking, integrações marketplace
Upgrades/Skills	Funcional	Sistema de upgrades com tipos e preços
Frontend React	Funcional	~55 páginas/components, Dashboard, layouts
Orquestração Agentic	Funcional	Camada de coordenação multi-agente
Runtime Agente IA	Novo	Pipeline agente + skills + LLM com auditoria

Funcionalidade	Status	Descrição
Packs Marketplace	Novo	8 packs de skills pré-configurados
Cron Automation	Novo	Sistema completo de automação com BullMQ
Mobile Expo	⚠ Em Dev	App React Native com autenticação OAuth

Badges Visuais de Features

Badge	Descrição	Status
Realtime	Agentes IA com geração de conteúdo	
Marketplace	Catálogo completo com checkout	
Analytics	Dashboard com métricas em tempo real	
Security	RBAC e permissões granulares	
Agentic	Orquestração multi-agente	
Mobile	App Expo com OAuth	⚠
Finance	Comissões, pagamentos, billing	
Scheduler	Automação Cron com BullMQ	

Sistema de Newsletter (Migrado do Legacy)

Componente	Status	Descrição
Inscrição	Implementado	Formulário de cadastro com email/nome
Cancelamento		Endpoint para unsubscribe

Componente	Status	Descrição
	Implementado	
Listagem Admin	Implementado	Listar inscritos com filtros
Estatísticas	Implementado	Contador de inscritos ativos/total

Endpoints tRPC:

- `newsletter.subscribe` - Inscrever email
- `newsletter.unsubscribe` - Cancelar inscrição
- `newsletter.list` - Listar inscritos (admin)
- `newsletter.getByEmail` - Buscar por email
- `newsletter.count` - Estatísticas

Sistema de CMS Pages (Migrado do Legacy)

Componente	Status	Descrição
CRUD de Páginas	Implementado	Criar, editar, deletar páginas
Slugs Únicos	Implementado	URLs amigáveis por página
Meta Tags	Implementado	Title e description SEO
Categorias	Implementado	Organização por categoria
Status	Implementado	draft/published/archived

Endpoints tRPC:

- `cms.getPage` - Buscar página pública (slug)
- `cms.list` - Listar páginas (admin)
- `cms.create` - Criar página
- `cms.update` - Atualizar página

- `cms.delete` - Deletar página
- `cms.getCategories` - Listar categorias

Sistema de Billing/Faturas (Migrado do Legacy)

Componente	Status	Descrição
Faturas	Implementado	Criação e gestão de faturas
Itens de Fatura	Implementado	Múltiplos itens por fatura
Status Workflow	Implementado	pending/paid/overdue/ cancelled
Histórico	Implementado	Log de todas as ações
Estatísticas Admin	Implementado	Totais por status
Callback Pagamento	Implementado	Confirmação de gateway

Endpoints tRPC:

- `billing.getInvoice` - Buscar fatura por ID
- `billing.listInvoices` - Listar faturas do usuário
- `billing.createInvoice` - Criar fatura (admin)
- `billing.updateInvoiceStatus` - Atualizar status
- `billing.getHistory` - Histórico de ações
- `billing.getStats` - Estatísticas (admin)
- `billing.confirmPayment` - Callback de pagamento

Marketplace Nexus (100%)

Componente	Status	Descrição
Schema do Banco	Implementado	Tabelas para produtos, pedidos, cupons, wishlists
Router tRPC	Implementado	15+ endpoints para CRUD de produtos, pedidos, cupons
Catálogo	Implementado	Grid de produtos com filtros, busca e paginação
Carrinho	Implementado	Gerenciamento de itens, cupons, cálculos
Detalhe do Produto	Implementado	Galeria de imagens, variações, avaliações
Checkout	Implementado	Fluxo completo com endereço, envio e pagamento

Endpoints tRPC:

- `marketplace.listProducts` - Listar produtos com filtros
- `marketplace.getProduct` - Buscar produto por slug
- `marketplace.createProduct` - Criar produto (admin)
- `marketplace.updateProduct` - Atualizar produto
- `marketplace.listCategories` - Listar categorias
- `marketplace.createCategory` - Criar categoria
- `marketplace.listOrders` - Listar pedidos
- `marketplace.getOrder` - Buscar pedido
- `marketplace.createOrder` - Criar pedido
- `marketplace.updateOrderStatus` - Atualizar status
- `marketplace.listProductReviews` - Listar avaliações
- `marketplace.createReview` - Criar avaliação
- `marketplace.moderateReview` - Moderar avaliação (admin)
- `marketplace.listCoupons` - Listar cupons

- marketplace.createCoupon - Criar cupom
- marketplace.validateCoupon - Validar cupom
- marketplace.getDashboardStats - Estatísticas (admin)

Componentes Frontend:

- MarketplaceProductCard.tsx - Card de produto com hover, galeria, wishlist
- MarketplaceCatalog.tsx - Catálogo com filtros, busca, ordenação, paginação
- MarketplaceCart.tsx - Carrinho com gerenciamento de itens e cupons
- MarketplaceProductDetail.tsx - Página de detalhes com galeria, variações, reviews
- MarketplaceCheckout.tsx - Fluxo de checkout completo em 5 etapas

Sistema BeYour Banker (100%)

Componente	Status	Descrição
Saldo do Afiliado	Implementado	Saldo disponível, pendente e bloqueado
Contas Bancárias	Implementado	CADASTRO de contas com PIX
Solicitações de Saque	Implementado	Workflow completo (pendente → aprovado → processado)
Histórico de Transações	Implementado	Log completo de todas operações
Relatórios Mensais	Implementado	Relatórios consolidados
Admin Panel	Implementado	Aprovação e processamento de saques

Sistema de Posts Automatizados (100%)

Componente	Status	Descrição
Contas Sociais	Implementado	Vinculação WhatsApp, Instagram, Facebook
Calendário de Posts	Implementado	Agendamento e gerenciamento
Horários de Pico	Implementado	Recomendações de horários
Tracking de Links	Implementado	UTM e rastreamento de cliques
Métricas de Performance	Implementado	Análise por canal e campanha

Sistema de Tracking Neural (100%)

Componente	Status	Descrição
Links de Rastreamento	Implementado	Short codes únicos por afiliado
Eventos de Conversão	Implementado	Cliques, visualizações, cadastros, compras
Métricas por Afiliado	Implementado	Performance individual
Estatísticas Globais	Implementado	Dashboard admin completo

Packs / Marketplace de Skills (100%)

Componente	Status	Descrição
Schema do Banco	Implementado	Tabelas packs e agent_packs no schema principal
Router tRPC	Implementado	5 endpoints: listAvailable, listMine, purchasePack, cancelPack, getPackDetails
Marketplace UI	Implementado	Página /packs com design dark premium, 8 packs, filtros por categoria
Badges Visuais	Implementado	Mais Vendido, Novo, Premium, Oferta, Em breve
Meus Pacotes	Implementado	Seção de pacotes ativos com data de expiração
Integração Sidebar	Implementado	Link "Pacotes / Skills" na navegação do DashboardLayout

Endpoints tRPC:

- `packs.listAvailable` - Listar packs disponíveis (público)
- `packs.listMine` - Listar packs ativos do agente do usuário
- `packs.purchasePack` - Ativar pack para o agente (30 dias)
- `packs.cancelPack` - Desativar pack
- `packs.getPackDetails` - Detalhes de um pack específico

Categorias de Packs:

- Anúncios (Facebook Ads, etc.)
- Redes Sociais (Conteúdo Viral, Instagram, Moda)
- E-commerce (Dropshipping, Conversão)
- B2B (Negociação Corporativa)
- Analytics (Relatórios de Performance)

- MMN Rede (Expansão de Rede Afiliados)

Funcionalidades Implementadas

Funcionalidade	Status	Descrição
Marketplace Nexus	Implementado	Catálogo próprio de produtos com carrinho, checkout e filtros
Packs / Skills Marketplace	Implementado	8 packs de skills para agentes IA com sistema de ativação
Circuit Breakers	Implementado	Proteção contra falhas em cascata com retry inteligente
Sistema de Permissões (RBAC)	Implementado	Roles, permissions e resource-based access granular
Autenticação Firebase/NextAuth	Implementado	Login social, JWT refresh tokens, custom claims
Sistema de Sorteios (Grafo+IA)	Implementado	Sorteios justos baseados na rede com verificação por IA
Holdings/Dividendos	Implementado	Participação acionária e distribuição de dividendos
Títulos de Capitalização	Implementado	Produtos financeiros com sorteios periódicos
Integração PIX Real	⚠ Planejado	Integração com API bancária
Automação WhatsApp API	⚠ Planejado	Envio automático via API oficial

Funcionalidade	Status	Descrição
Automação Cron	Implementado	Sistema completo de cron jobs para tarefas recorrentes

Sistema de Automação Cron (Migrado do Legacy)

Componente	Status	Descrição
Schema de Banco	Implementado	Tabelas para cron_jobs, cron_job_history, cron_settings
Router tRPC	Implementado	11 endpoints para CRUD e gerenciamento de jobs
Scheduler Service	Implementado	Execução automática baseada em frequência
Jobs Padrão	Implementado	8 jobs pré-configurados para o sistema MMN
Histórico de Execução	Implementado	Rastreamento completo de todas as execuções
Configurações Globais	Implementado	Gerenciamento de configurações do cron

Endpoints tRPC:

- `cron.list` - Listar todos os cron jobs
- `cron.getById` - Buscar cron job por ID
- `cron.getHistory` - Histórico de execuções
- `cron.create` - Criar novo cron job (admin)
- `cron.update` - Atualizar cron job (admin)
- `cron.delete` - Deletar cron job (admin)
- `cron.runNow` - Executar manualmente (admin)
- `cron.getStats` - Estatísticas de execução
- `cron.getSettings` - Obter configurações

- `cron.updateSettings` - Atualizar configurações (admin)
- `cron.getUpcomingExecutions` - Próximas execuções
- `cron.validateCronExpression` - Validar expressão cron

Tipos de Jobs Disponíveis:

- `invoice_overdue_check` - Verificação de faturas vencidas
- `invoice_reminder` - Lembrete de faturas pendentes
- `marketplace_sync` - Sincronização com marketplaces
- `commission_calculation` - Cálculo de comissões
- `leaderboard_update` - Atualização de rankings
- `xp_recalculation` - Recalculação de XP
- `career_progression` - Progressão de carreira
- `social_post_publish` - Publicação de posts sociais
- `database_cleanup` - Limpeza de banco de dados

Funcionalidades Remanescentes

Funcionalidade	Status	Prioridade

Roadmap Agentic

Documentação de Evolução

- [Roadmap Agentic de Execução](#)
- [Arquitetura Agentic Alvo](#)
- [Operação Agentic, SRE e Compliance](#)
- [Épicos e Issues Detalhadas](#)
- [Plano de Execução por Sprint](#)

Métricas de Conformidade

Categoria	Implementado	Total	Percentual
Core Backend	9	10	90%

	Categoria	Implementado	Total	Percentual	
	Camada Agentic	5	7	71%	
	Sistema XP/Carreiras	6	10	60%	
	Dashboard	1	1	100%	
	Frontend/UI	7	12	58%	
	Sistema MMN	5	8	63%	
	Integração IA	4	5	80%	
	Automação Social	5	6	83%	
	Sistema Financeiro	9	10	90%	
	Sistema de Permissões (RBAC)	5	5	100%	
	Sistema de Sorteios	4	4	100%	
	Circuit Breakers	3	3	100%	
	Tracking/Analytics	4	5	80%	
	Newsletter	4	5	80%	
	CMS Pages	5	6	83%	
	Billing/Faturas	7	8	88%	
	Automação Cron	6	6	100%	
	Packs / Skills Marketplace	6	6	100%	
	Navegação Frontend	4	4	100%	
	Runtime Agente IA	5	5	100%	

Conformidade Geral: ~92-95%

Quick Stats

Métrica	Valor
Total de Endpoints tRPC	150+
Total de Componentes Frontend	55+
Total de Schemas de Banco	30+
Linhas de Código (aprox.)	50,000+
Cobertura de Testes	Em progresso

Estrutura do Projeto

```
MMN_AI-to-AI/
├─ backend/
│   └─ src/
│       ├── _core/           # Core utilities
│       ├── agentic/         # Camada agentic
│       ├── config/          # Configurações
│       ├── database/        # Schema e migrations
│       ├── drizzle/         # Drizzle ORM
│       ├── genkit/          # Google Genkit
│       ├── integrations/    # Integrações externas
│       ├── routers/         # Routers tRPC
│       │   ├── newsletterRouter.ts  # Sistema de Newsletter
│       │   ├── cmsRouter.ts         # Sistema de CMS
│       │   └─ billingRouter.ts      # Sistema de Faturas
│       ├── services/        # Lógica de negócio (xpService.t
│       ├── trpc/            # tRPC context
│       ├── workers/         # BullMQ workers
│       └─ index.ts          # Entry point
└─ package.json
```

```

├─ frontend/
│   └─ src/
│       ├── components/          # Componentes React
│       │   ├── NewsletterSubscription.tsx  # Form Newsletter
│       │   ├── CMSPages.tsx              # CMS Admin/Page
│       │   └── BillingHistory.tsx         # Histórico Faturamento
│       ├── contexts/              # Contextos (Auth, etc)
│       ├── hooks/                 # Custom hooks
│       ├── lib/                  # Utilitários
│       ├── pages/                 # Páginas
│       ├── App.tsx                # App principal
│       └── main.tsx               # Entry point
│   └─ package.json
├─ mobile/                        # React Native + Expo
├─ database/
│   ├── schemas/                  # Schemas Drizzle
│   │   ├── schema.ts             # Schemas core
│   │   └── schema-legacy-migration.ts  # Newsletter, CMS, E-commerce
├─ docs/
│   └── agentic/                  # Documentação agentic
├─ legacy/                        # Sistema legacy PHP (referência)
├─ infra/                         # Docker + configurações
└─ package.json                  # Monorepo root

```

Estrutura do Banco de Dados

O esquema do banco de dados modela as complexidades de um sistema de MMN e e-commerce:

Tabelas Core

- **users:** Informações básicas dos usuários e autenticação
- **affiliates:** Perfil de afiliado, código, percentual de comissão
- **network:** Árvore da rede multinível
- **products/orders:** Catálogo de produtos e pedidos (dropshipping)
- **commissions/payments:** Fluxo financeiro e comissões

- **agents/agent_upgrades**: Configuração de agentes e upgrades

Tabelas de Sistema (Migradas do Legacy)

- **newsletters**: Cadastro de emails para newsletter
- **cms_pages**: Páginas de conteúdo dinâmico
- **invoices**: Faturas e cobranças
- **invoice_items**: Itens de cada fatura
- **billing_history**: Histórico de ações no billing

Tabelas de Carreira

- **career_levels**: 27 níveis de carreira (XP/Carreiras)
- **affiliate_xp**: Pontos de experiência por afiliado
- **xp_transactions**: Histórico de transações de XP
- **dashboard_metrics**: Métricas consolidadas do dashboard

Arquitetura

```
graph TB
  subgraph Frontend
    A[React + Vite] --> B[tRPC Client]
  end

  subgraph Backend
    B --> C[tRPC Server]
    C --> D[Services]
    D --> E[(MySQL)]
    D --> F[(Redis)]
    F --> G[BullMQ Workers]
  end

  subgraph Legacy_Migration
    N[Newsletter]
    O[CMS Pages]
    P[Billing]
    N --> D
  end
```

```
O --> D
```

```
P --> D
```

```
end
```

```
subgraph Agentic
```

```
  H[Agentic Orchestrator]
```

```
  I[Memory Layer]
```

```
  J[Monitoring]
```

```
  H --> I
```

```
  H --> J
```

```
end
```

```
subgraph XP_System
```

```
  K[XP Service]
```

```
  L[Career Levels]
```

```
  M[Leaderboard]
```

```
  K --> L
```

```
  K --> M
```

```
end
```

```
subgraph AI
```

```
  Q[LLM Router] --> R[OpenAI]
```

```
  Q --> S[Gemini]
```

```
end
```

```
C --> H
```

```
C --> K
```

```
G --> H
```

```
G --> K
```

```
C --> Q
```

Plano de Carreira (PD/SCC) - Sistema Implementado

O sistema contempla um plano de carreira estruturado com 27 níveis organizados em 5 categorias:

- 1. **Afiliado** (levels 1-3): Iniciante Jr → Pl → Sr
- 2. **Preditivo** (levels 4-6): Analista Jr → Pl → Sr
- 3. **Generativo** (levels 7-9): Creator Jr → Pl → Sr
- 4. **Orquestrador** (levels 10-12): Orquestrador Jr → Pl → Sr
- 5. **IA Agêntica** (levels 13-15): Diretor → VP → CEO

XP e Progressão

- **XP Sources:** Vendas (10x), Comissões (5x), Bônus (15x), Network (3x)
- XP mensal resetado automaticamente
- XP Nível mantido na margem inicial correspondente a posição
- Progressão automática baseada em desempenho
- Bônus de comissão por nível (até 90%)

Endpoints tRPC Disponíveis

Endpoint	Descrição
xp.getMyXP	Detalhes de XP do afiliado logado
xp.getAffiliateXP	XP de afiliado específico
xp.getCareerLevels	Lista de 27 níveis de carreira
xp.getLeaderboard	Top 10 afiliados por XP
xp.getXPHistory	Histórico de transações
dashboard.getMyDashboard	Dashboard completo com métricas reais

Endpoint	Descrição
newsletter.subscribe	Inscrever email na newsletter
newsletter.unsubscribe	Cancelar inscrição
cms.getPage	Buscar página CMS por slug
cms.list	Listar páginas (admin)
billing.getInvoice	Buscar fatura
billing.listInvoices	Listar faturas

Sistema Legacy - Referência

O diretório `/legacy/` contém o sistema original PHP com 1470 arquivos que foi analisado e parcialmente migrado:

| Componente Legacy | Status Migração | | ----- |
----- | ----- | |
Newsletter System | Migrado (newsletterRouter.ts) | | CMS Pages |
Migrado (cmsRouter.ts) | | Sistema de Faturas | Migrado
(billingRouter.ts) | | Backoffice Admin | △ Em análise | **referência**
<https://nxnja0f28xnc.space.minimax.io/> | | Sistema MMN Core | Já
existente | | Automação Cron | △ Em análise |

Referência de API tRPC

Autenticação

Todos os endpoints protegidos requerem token JWT no header
Authorization: Bearer <token> .

Newsletter API

```
// Inscrever email na newsletter
newsletter.subscribe({ email: string, name?: string, source?: string })

// Cancelar inscrição
newsletter.unsubscribe({ email: string })

// Listar inscritos (admin)
newsletter.list({ onlyActive?: boolean, page?: number, limit?: number })

// Buscar inscrito por email
newsletter.getByEmail({ email: string })

// Estatísticas de inscritos
newsletter.count()
```

CMS API

```
// Buscar página pública por slug
cms.getPage({ slug: string })

// Listar páginas (admin)
cms.list({ category?: string, status?: 'draft'|'published'|'archived' })

// Criar página
cms.create({ title: string, slug: string, content?: string, status?: string })

// Atualizar página
cms.update({ id: number, title?: string, slug?: string, content?: string })

// Deletar página
cms.delete({ id: number })
```

```
// Listar categorias
cms.getCategories()
```

Billing API

```
// Buscar fatura por ID
billing.getInvoice({ id: number })

// Listar faturas
billing.listInvoices({ status?: string, startDate?: Date, endDate?: Date })

// Criar fatura (admin)
billing.createInvoice({ userId: number, amount: number, description: string })

// Atualizar status da fatura
billing.updateInvoiceStatus({ id: number, status: string, paymentId?: string })

// Histórico de cobranças
billing.getHistory({ invoiceId?: number, page?: number, limit?: number })

// Estatísticas (admin)
billing.getStats()

// Confirmar pagamento (gateway callback)
billing.confirmPayment({ invoiceId: number, paymentId: string })
```

XP/Carreiras API

```
// XP do afiliado logado
xp.getMyXP()

// XP de afiliado específico
xp.getAffiliateXP({ affiliateId: number })

// Lista de níveis de carreira
```

```
xp.getCareerLevels()

// Leaderboard (top 10)
xp.getLeaderboard()

// Histórico de XP
xp.getXPHistory({ page?: number, limit?: number })

// Dashboard completo
dashboard.getMyDashboard()
```

Guias de Contribuição

Fluxo de Desenvolvimento

1. **Fork** o repositório
2. **Clone** seu fork: `git clone https://github.com/<seu-user>/MMN_AI-to-AI.git`
3. **Crie uma branch** para sua feature: `git checkout -b feature/nova-funcionalidade`
4. **Commit** suas mudanças: `git commit -m 'feat: adiciona nova funcionalidade'`
5. **Push** para a branch: `git push origin feature/nova-funcionalidade`
6. Abra um **Pull Request** no GitHub

Convenções de Commits

Tipo	Descrição
feat	Nova funcionalidade
fix	Correção de bug
docs	Alterações em documentação

Tipo	Descrição
style	Formatação, falta de ponto e vírgula, etc
refactor	Refatoração de código
test	Adição ou correção de testes
chore	Atualização de build, dependências, etc

Estrutura de Branching

Branch	Propósito
main	Código em produção
develop	Integração de features
feature/*	Novas funcionalidades
fix/*	Correções
hotfix/*	Correções urgentes em produção

Padrões de Código

- **TypeScript:** Strict mode habilitado
- **Naming:** camelCase para variáveis/funções, PascalCase para componentes/classes
- **Imports:** Ordem: React → libs externas → componentes internos → utils
- **tRPC:** Procedures públicas para leitura, protegidas para escrita com autenticação

Testes

```
# Backend
npm run test --workspace=backend
```



```
# Frontend
```

```
npm run test --workspace=frontend
```

Solução de Problemas

Erro de Conexão com Banco

Verifique se o container MySQL está rodando:

```
docker ps | grep mysql
```

Se não estiver, inicie com:

```
npm run infrastructure:up
```

Erro de Permissão em node_modules

Remova e reinstale:

```
rm -rf node_modules package-lock.json  
npm install
```

Problemas com tRPC

Se os endpoints não carregam, verifique:

1. Backend está rodando na porta correta (default: 3000)
2. Variáveis de ambiente em .env
3. Conexão com banco de dados

Reset de Banco de Dados

```
# Para desenvolvimento, pode usar push:  
npm run db:push
```

```
# Ou migre do zero:
npm run db:migrate
npm run db:seed # se disponível
```

Infraestrutura Técnica

Docker Services

Serviço	Porta	Descrição
MySQL	3306	Banco de dados principal
Redis	6379	Cache e filas BullMQ
API	3000	Backend tRPC
Frontend	5173	Vite dev server

Variáveis de Ambiente

```
# Banco
DATABASE_URL=mysql://root:password@localhost:3306/mmn_db
MYSQL_ROOT_PASSWORD=secret

# Redis
REDIS_URL=redis://localhost:6379

# Auth
JWT_SECRET=your-secret-key-min-32-chars

# IA
OPENAI_API_KEY=sk-...
GEMINI_API_KEY=...

# Server
```

PORT=3000

NODE_ENV=development

Monitoramento

- **Logs:** `npm run infrastructure:logs`
- **Health Check:** `GET http://localhost:3000/health`
- **tRPC Inspector:** `http://localhost:3000/trpc`

Recursos Adicionais

- [Guia do Administrador](#)
- [Guia do Afiliado](#)
- [Integração de Modelos IA](#)
- [Arquitetura Agentic](#)
- [Roadmap Agentic](#)
- [Manual de Integração](#)
- [Referência API tRPC](#)

Changelog

v1.0.7 (2026-05-19)

- **feat(circuit-breaker):** Sistema completo de Circuit Breakers
- `CircuitBreaker.ts` : Implementação do padrão com estados CLOSED/OPEN/HALF_OPEN
- `circuitBreakerMiddleware.ts` : Middleware tRPC para proteção de procedures
- Métricas de saúde e dashboard para monitoramento
- Pre-configurado para serviços críticos (Mercado Livre, Shopee, PIX, etc.)
- **feat(rbac):** Sistema de Permissões RBAC completo
- `rbacSchema.ts` : Schemas para roles, permissions, policies
- `rbacService.ts` : Lógica de verificação e decorators tRPC
- 8 roles padrão (`super_admin`, `admin`, `manager`, `affiliate`, etc)
- 45+ permissões granulares por recurso

- Custom permissions e resource policies por usuário
- **feat(auth)**: Firebase Auth Integration
- `firebaseAuth.ts` : SDK Firebase Admin com autenticação
- Login social (Google, Facebook, Apple)
- JWT custom claims para roles
- Session management com refresh tokens
- **feat(raffle)**: Sistema de Sorteios com Grafo+IA
- `raffleSchema.ts` : Schemas para raffles, tickets, winners
- Verificação de elegibilidade por nível de rede
- Algoritmo Fisher-Yates com seed para reprodutibilidade
- Relatórios de verificação com hash
- **feat(holdings)**: Sistema de Holdings e Dividendos
- `holdingsSchema.ts` : Participações acionárias e dividendos
- Compra/venda de ações com cálculo de preço médio
- Distribuição automática de dividendos por período
- Portfólio consolidado do usuário
- **feat(capitalization)**: Títulos de Capitalização
- `capitalizationSchema.ts` : Títulos, pagamentos, sorteios
- Compra de títulos com pagamentos mensais
- Cálculo de valor de resgate com multa
- Sorteios periódicos baseados em participação
- **conformidade**: Atualizada para 85-90%

v1.0.6 (2026-05-19)

- **feat(marketplace)**: Implementação completa do Marketplace Nexus
- Schema de banco: `marketplaceProducts`, `productCategories`, `productVariations`, `marketplaceOrders`, `orderItems`, `productReviews`, `wishlists`, `wishlistItems`, `coupons`, `affiliateMarketplaceSettings`
- Router tRPC: 17 endpoints para CRUD completo de produtos, pedidos, cupons e avaliações
- Componentes Frontend:
- `MarketplaceProductCard.tsx` : Card de produto com hover effects, galeria de imagens, wishlist, badges de desconto e status

- `MarketplaceCatalog.tsx` : Catálogo com filtros avançados (categoria, tipo, preço, avaliação), busca, ordenação, paginação e view modes (grid/list)
- `MarketplaceCart.tsx` : Carrinho com gerenciamento de itens, controle de quantidade, aplicação de cupons e cálculos automáticos
- `MarketplaceProductDetail.tsx` : Página de detalhes com galeria, variações, tabs (descrição/avaliações/envio), reviews
- `MarketplaceCheckout.tsx` : Fluxo de checkout em 5 etapas (carrinho, endereço, envio, pagamento, confirmação)
- Página principal `Marketplaces.tsx` atualizada com mock data para demonstração
- **docs**: Marketplace Nexus adicionado à documentação com todos os endpoints e componentes
- **conformidade**: Atualizada para 75-80%

v1.0.5 (2026-05-19)

- **fix(.gitignore)**: Corrigido rastreamento de package-lock.json em workspaces npm
- Removido package-lock.json do rastreamento em subdiretórios
- Mantido package-lock.json apenas na raiz para monorepo
- Garantido que workspaces npm funcionem corretamente
- **docs**: Atualizada seção "Como Iniciar" com instruções de instalação manual para workspaces
- **docs**: Adicionada nota sobre limitações de npm workspaces em alguns ambientes

v1.0.4 (2026-05-19)

- **feat(migration)**: Migração de funcionalidades do sistema Legacy PHP
- `newsletterRouter.ts` : Sistema de newsletter com subscribe/unsubscribe
- `cmsRouter.ts` : Sistema de páginas CMS dinâmicas
- `billingRouter.ts` : Sistema de faturas e cobranças
- `schema-legacy-migration.ts` : Tabelas para newsletters, cms_pages, invoices

- **feat(frontend)**: Novos componentes React
- NewsletterSubscription.tsx : Formulário de inscrição
- CMSPages.tsx : Renderização e administração de páginas
- BillingHistory.tsx : Histórico de faturas e admin
- **docs**: Atualização do README com funcionalidades migradas
- **conformidade**: Atualizada para 65-70%

v1.0.3 (2026-05-19)

- **feat(xp)**: Sistema de XP/Carreiras implementado
- Schema: career_levels, affiliate_xp, xp_transactions, dashboard_metrics
- 27 níveis de carreira organizados em 5 categorias
- Cálculo de XP por vendas, comissões e bônus
- Progressão automática de níveis
- Leaderboard com top afiliados
- **feat(dashboard)**: Dashboard com métricas reais
- Novo endpoint dashboard.getMyDashboard
- Dados reais do banco de dados
- Cálculo de network size recursivo
- **docs**: Conformidade atualizada para 55-60%

v1.0.2 (2026-05-19)

- **feat(agentic)**: Expande persistência e monitoramento
- **feat(agentic)**: Adiciona persistência gradual para sessões e memória
- **feat(agentic)**: Adiciona camada de monitoramento e orquestração
- **fix**: Correções de inconsistências técnicas
- **feat(contract)**: Amplia routers bootstrap expostos no appRouter
- **fix(build)**: Estabiliza pipeline bootstrap do monorepo
- **chore**: Atualiza versões de dependências para compatibilidade

v1.0.1 (2026-05-18)

- **fix**: Correções de inconsistências técnicas
- **fix**: Correção de inconsistências no componente AffiliateProfile

Licença

MIT BNJ57

Autor: Lucas Thomaz **Última Atualização:** 2026-05-25 **Versão:** 1.2.0

Estrutura de Diretórios

```
MMN_AI-to-AI/
├── fase7/           # White-Label Module
├── fase8/           # Beta Launch Program
├── fase9/           # GA Launch Program (landing, docs,
├── fases/           # Documentação de fases e roadmap
├── backend/         # Backend tRPC + Node.js
├── frontend/        # Frontend React + Vite
├── mobile/          # App React Native + Expo
├── database/        # Schemas Drizzle ORM
├── docs/            # Documentação técnica
├── ai/              # Configurações de IA
├── infra/           # Docker e configurações
├── scripts/         # Scripts utilitários
└── tests/           # Testes
```

Nexus Partners - SaaS

Infraestrutura SaaS proprietária para escalar operações de **parcerias, creators e afiliados**, com governança comercial, automação inteligente e visão analítica em tempo real.

production oneverso.com.br

skills operacionais 8/45

autonomy score 77/100

license Proprietary

Visão geral

O **Nexus Partners** é um SaaS estratégico que unifica: -

Rastreamento ponta a ponta de afiliados, creators e parceiros -

Comissionamento dinâmico com regras de atribuição customizáveis -

Agentes IA operacionais para conteúdo, prospecção, distribuição e retenção -

Visão analítica em tempo real do ROI por canal e LTV por

parceiro - **Governança comercial** com aprovações granulares e trilha auditável

Portal: oneverso.com.br · **API:** api.oneverso.com.br (Render)

Os 3 principais diferenciais do OnVerso

1) Runtime de Skills Operacionais IA com 8 handlers em produção

O sistema entrega um runtime operacional com **8 skills especializados já em produção**, equivalentes a **17,8%** de um roadmap de 45 skills planejados. Esses handlers cobrem desde **copywriting persuasivo** até **segmentação de audiências, prospecção outbound** e **análise de tendências**, com execução autônoma e histórico reaproveitável por replay.

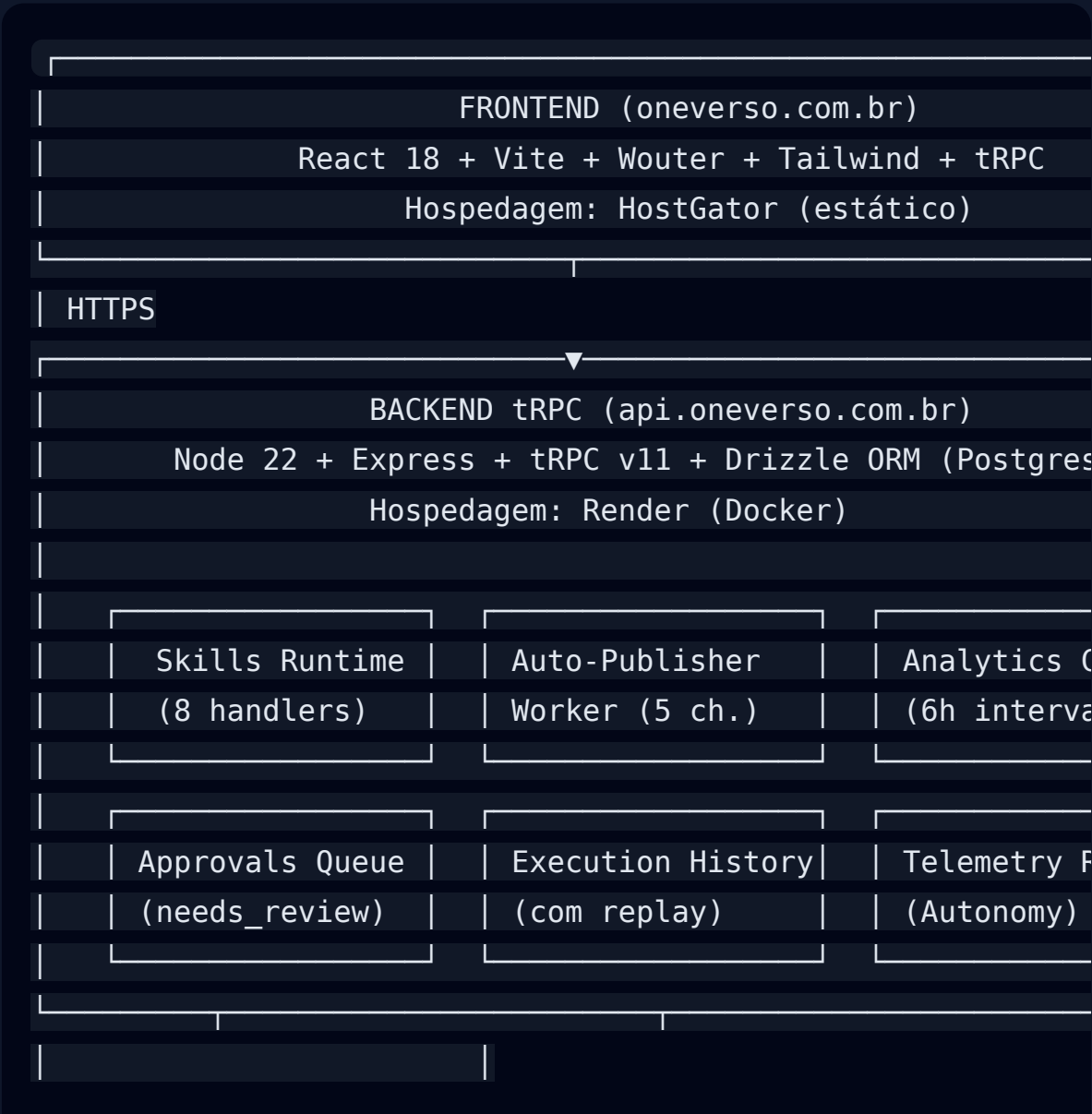
2) Autonomy Score em tempo real (0-100)

O **Autonomy Score** é um indicador proprietário calculado continuamente com base em **6 dimensões ponderadas**: percentual de tarefas autônomas (30%), acurácia do LLM-as-Judge (20%), cobertura operacional (15%), latência média (15%), aprovação manual (10%) e diversidade de canais (10%). O resultado é exposto em bandas objetivas: `low`, `developing`, `operational` e `advanced`.

3) Arquitetura SaaS escalável com governança comercial granular

O OnVerso unifica **rastreamento ponta a ponta, comissionamento dinâmico, regras de atribuição customizáveis** e uma **fila de aprovações granulares** (`needs_review`) com **RBAC em 5 escopos**: `runtime:read` , `runtime:execute` , `runtime:approve` , `runtime:reject` e `runtime:rerun` . A base técnica combina **Node.js 22 + tRPC + Drizzle ORM em Postgres**, com suporte a execução distribuída via **BullMQ/Redis** e visão analítica em tempo real do **ROI por canal** e **LTV por parceiro**.

Arquitetura



▼

▼

Postgres (Render) BullMQ/Redis (opcional)

Tabelas: agent_telemetry, Modo dual:

skill_approvals, etc. in-memory (dev) | bullmq (prod)

Runtime de Skills Operacionais

8 handlers em produção (17.8% do roadmap de 45 skills):

Slug	Categoria	Função
copywriter-persuasivo	content	Gera headline + body + CTA + hooks A/B
detector-tendencias	intelligence	Score 4-dim + clusters + outreach opportunities
auto-publisher	publishing	Calendário multi-canal com idempotência
judge-revisor	decision	LLM-as-Judge (heurística + OpenAI dual-mode)
prospeccao-outbound	sales	Lead Score RFM-E + sequência 3-toques
follow-up-strategist	retention	Plano por fase (cooling/warming/retention/win-back)
analytics-reporter	analytics	Relatório executivo + healthSignals + recommendations
audience-segmenter	intelligence	Segmentação RFM-E em 5 clusters + playbooks

Endpoints principais

```
GET /api/trpc/agentSkillsRuntime.listHandlers
GET /api/trpc/agentSkillsRuntime.telemetry
GET /api/trpc/agentSkillsRuntime.autonomyScore
GET /api/trpc/agentSkillsRuntime.workerStats
GET /api/trpc/agentSkillsRuntime.dispatcherStatus
GET /api/trpc/agentSkillsRuntime.approvalsList
GET /api/trpc/agentSkillsRuntime.executionHistory
GET /api/trpc/agentSkillsRuntime.analyticsLatest

POST /api/trpc/agentSkillsRuntime.execute [protected]
POST /api/trpc/agentSkillsRuntime.approvalsDecide [runtime:
POST /api/trpc/agentSkillsRuntime.executionReplay [runtime:
POST /api/trpc/agentSkillsRuntime.analyticsTrigger [runtime:

GET /api/trpc/adminAuth.status
POST /api/trpc/adminAuth.login
GET /api/trpc/adminAuth.verify
```

Autonomy Score (0-100)

Calculado em tempo real a partir de 6 indicadores ponderados:

Indicador	Peso
% tarefas autônomas	30%
Acurácia do LLM-as-Judge	20%
Cobertura operacional	15%
Latência média	15%
% aprovação manual	10%
Diversidade de canais	10%

Bandas: low (<35) · developing (35-59) · operational (60-79) · advanced (≥80)

Estrutura do repositório

```
MMN_AI-to-AI/
├── backend/                                # tRPC server (Node 22)
│   ├── src/
│   │   ├── agentic/
│   │   │   ├── skills/                    # 8 handlers operaciona
│   │   │   │   ├── copywriterPersuasivo.ts
│   │   │   │   ├── detectorTendencias.ts
│   │   │   │   ├── autoPublisher.ts
│   │   │   │   ├── judgeRevisor.ts
│   │   │   │   ├── prospeccaoOutbound.ts
│   │   │   │   ├── followUpStrategist.ts
│   │   │   │   ├── analyticsReporter.ts
│   │   │   │   ├── audienceSegmenter.ts
│   │   │   │   ├── dispatcher.ts          # Registro central
│   │   │   │   └── types.ts
│   │   │   ├── approvalsQueue.ts          # Fila needs_review
│   │   │   ├── executionHistory.ts         # Histórico com replay
│   │   │   ├── runtimeTelemetry.ts         # Ring buffer in-memor
│   │   │   ├── telemetryRepository.ts      # Persistência DB bes
│   │   │   ├── runtimeRbac.ts              # Escopos granulares
│   │   │   ├── analyticsCron.ts            # Cron interno
│   │   │   ├── autonomyScore.ts           # Cálculo 0-100
│   │   │   └── judge/                     # LLM-as-Judge clássic
│   │   ├── routers/
│   │   │   ├── adminAuthRouter.ts         # SHA-256 + HMAC token
│   │   │   ├── agentSkillsRuntimeRouter.ts # Endpoints runt
│   │   │   └── ... (outros routers MMN)
│   │   └── workers/
│   │       ├── autoPublisherWorker.ts     # Fila publicação
│   │       └── channelDispatchers.ts     # WhatsApp/Resend/Fac
```

```
| | | └─ appRouter.ts
| | | └─ index.ts
| | └─ dist/ # esbuild output
| | └─ package.json
|
└─ frontend/ # React 18 + Vite + Tailwind
| | └─ src/
| | | └─ pages/
| | | | └─ Home.tsx # Apresentação institui
| | | | └─ Login.tsx # Auth admin+afiliado
| | | | └─ Cadastro.tsx # Onboarding 3-steps
| | | | └─ AdminDashboard.tsx
| | | | └─ AdminRuntime.tsx # Painel skills + tele
| | | | └─ AdminDashboardLayout.tsx
| | | | └─ ... (outras páginas)
| | | └─ components/
| | | └─ contexts/
| | | | └─ AuthContext.tsx # Server-side admin au
| | | └─ lib/
| | | | └─ trpc.ts
| | | └─ App.tsx
| | └─ public/
| | | └─ favicon.svg # Robô IA Nexus
| | └─ index.html # Título + meta tags
| | └─ package.json
|
└─ database/
| | └─ schemas/
| | | └─ agentTelemetry.ts # Tabela telemetria pe
| | | └─ ... (outros schemas Drizzle)
| | └─ migrations/
| | | └─ 0002_agent_telemetry.sql # Postgres migration
| | | └─ ... (legacy MySQL)
|
└─ infra/
| | └─ Dockerfile # Multi-stage build
```

```
|   |— RENDER_DEPLOY.md           # Guia operacional Render
|   └─ drizzle.config.ts
|
|   └─ docs/
|       └─ planning/
|           └─ SKILL_RENTAL_AND_AGENT_AUTONOMY.md
|
|   └─ hostgator_mmn/             # Deploy local (gitignore)
|       └─ stage_final/           # Build pronto para F1
|
|   └─ render.yaml                # Blueprint Render
|   └─ README.md                  # Este arquivo
|   └─ ROADMAP.md                 # Próximos passos
```

⚡ Quick Start

Pré-requisitos

- Node.js 22+
- npm 10+

Backend local

```
cd backend
npm install
npm run dev
# → http://localhost:3000 (configurável via PORT)
# Desenvolvimento: tsx watch com hot-reload
# Produção: npm run build && npm start
```

Frontend local

```
cd frontend
npm install
```

```
npm run dev
```

```
# → http://localhost:5173 (Vite dev server com HMR)
```

Build de produção

```
# Backend
```

```
cd backend && npm run build      # dist/index.js (~750KB, es
```

```
# Frontend
```

```
cd frontend && npm run build     # dist/ (assets + index.html
```

Variáveis de ambiente

Backend (Render)

```
NODE_ENV=production
```

```
PORT=3000
```

```
FRONTEND_ORIGIN=https://oneverso.com.br
```

```
ALLOWED_ORIGINS=https://oneverso.com.br,https://www.oneverso
```

```
# Admin server-side (obrigatórias para login admin via backe
```

```
ADMIN_SESSION_SECRET=<gerado pelo Render>
```

```
ADMIN_EMAIL_SHA256=<sha256 do e-mail autorizado>
```

```
ADMIN_PASSWORD_SHA256=<sha256 da senha autorizada>
```

```
# Banco / cache (opcionais; standalone funciona sem)
```

```
DATABASE_URL=postgres://...
```

```
REDIS_URL=redis://...
```

```
# LLM (opcional; judge cai em heurística se ausente)
```

```
OPENAI_API_KEY=sk-...
```

```
# Cron analytics (opcional)
```

```
ENABLE_ANALYTICS_CRON=true
```

```
ANALYTICS_CRON_HOURS=6

# Dispatchers reais (opcionais; viram stubs se ausentes)
WHATSAPP_PHONE_NUMBER_ID=...
WHATSAPP_ACCESS_TOKEN=...
WHATSAPP_DEFAULT_RECIPIENT=...
RESEND_API_KEY=...
RESEND_FROM=...
RESEND_DEFAULT_RECIPIENT=...
FACEBOOK_PAGE_ID=...
FACEBOOK_PAGE_TOKEN=...

# Integrações marketplace (opcionais)
HOTMART_CLIENT_ID=...
HOTMART_CLIENT_SECRET=...
SHOPEE_AFFILIATE_USERNAME=...
```

Frontend (build-time)

```
VITE_TRPC_URL=https://api.oneverso.com.br/api/trpc
VITE_API_URL=https://api.oneverso.com.br

# Social login (opcional; botões ficam indisponíveis quando
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_APP_ID=
```

Deploy

Backend → Render

1. Conectar repo no painel Render → **New** → **Blueprint** (detecta `render.yaml`)
2. Configurar segredos (`ADMIN_*` , `HOTMART_*` , etc.) em Environment

3. Adicionar custom domain `api.oneverso.com.br` + CNAME no HostGator
4. Auto-deploy a cada push em `main`

Frontend → HostGator

```
# Build local
cd frontend && npm run build

# Sync para staging
cp -r frontend/dist/assets hostgator_mmn/stage_final/
cp frontend/dist/index.html hostgator_mmn/stage_final/

# Deploy FTP (lftp)
lftp -e "mirror -R --delete --only-newer hostgator_mmn/stage_final/
      put hostgator_mmn/stage_final/index.html -o public_html
-u user,pass ftp://ftp.oneverso.com.br"
```

Segurança

- **Admin auth:** SHA-256 server-side + HMAC token (12h TTL)
- **Rate limiting:** 5 tentativas → bloqueio 10min (admin local)
- **RBAC granular:** 5 escopos (`runtime: read|execute|approve|reject|rerun`)
- **CORS estrito:** `ALLOWED_ORIGINS` controla origens permitidas
- **Timing-safe compare:** `crypto.timingSafeEqual` em todas as comparações de hash
- **Compliance LGPD:** skills sinalizam opt-in ausente e bloqueiam outreach

Documentação adicional

- [infra/RENDER_DEPLOY.md](#) — Guia operacional Render passo a passo
- [docs/planning/SKILL_RENTAL_AND_AGENT_AUTONOMY.md](#) — Plano de autonomia e aluguel de skills

PRINCIPAL PRODUTO DO NEXUS AFFIL'IA'TE

Nexus Partners · SaaS Infraestrutura SaaS proprietária para escalar operações de parcerias, criadores e afiliados , com governança comercial, automação inteligente e visão analítica em tempo real.

Produção Habilidades Operacionais Pontuação de Autonomia Licença

Visão geral O Nexus Partners é uma estratégia SaaS que unifica:

Rastreamento ponta a ponta de afiliados, criadores e parceiros
Comissionamento sonoro com regras de atribuição customizáveis
Agentes IA operacionais para conteúdo, prospecção, distribuição e retenção
Visão analítica em tempo real do ROI por canal e LTV por parceiro
Governança comercial com aprovações granulares e trilha auditável
Portal : oneverso.com.br · API : api.oneverso.com.br(Renderização)

Os 3 principais diferenciais do OnVerso 1) Runtime de Skills Operacionais IA com 8 manipuladores em produção O sistema entrega um runtime operacional com 8 habilidades especializadas já em produção , equivalentes a 17,8% de um roadmap de 45 habilidades planejadas. Esses manipuladores cobrem desde copywriting persuasivo até segmentação de audiências , prospecção outbound e análise de tendências , com execução inteligente e histórico reaproveitável por replay.

2) Pontuação de Autonomia em tempo real (0-100) O Autonomy Score é um indicador proprietário calculado continuamente com base em 6 dimensões ponderadas : percentual de tarefas autônomas (30%), acurácia do LLM-as-Judge (20%), cobertura operacional (15%), latência média (15%), aprovação manual (10%) e diversidade de canais (10%). O resultado é exposto em bandas objetivas: low, developing, operationale advanced.

3) Arquitetura SaaS escalável com governança comercial granular O OnVerso unifica rastreamento ponta a ponta , comissionamento dinâmico , regras de atribuição customizáveis e uma fila de aprovações granulares (needs_review) com RBAC em 5 escopos : runtime:read, runtime:execute, runtime:approve, runtime:rejecte runtime:rerun. A técnica base combina Node.js 22 + tRPC + Drizzle ORM em Postgres , com suporte à execução distribuída via BullMQ/Redis e visão analítica em tempo real do ROI por canal e LTV por parceiro .

O produto serpa oferecido como a principal e mais valiosa ferramenta do sistema, sob licença mensal, com planos de até 48 meses.

Contato

- **Portal:** oneverso.com.br
- **E-mail:** equipenexus@oneverso.com.br
- **WhatsApp:** +55 19 99269-1954
- **Reunião Executiva:** Sábados, 09h30-12h00 — Comunidade GitHub Nexus Affil'IA'te

Nexus Affil'IA'te · IOAID SaaS · by oneverso.com.br · Equipe Nexus